Introduction to Computer Programming

Prof. Alessio Martino

amartino@luiss.it

Department of Business and Management
Viale Romania 32, 00197 Rome

A.Y. 2024-2025

Outline

1. Introduction
   - Indices and Ordering
2. Mutability
3. Iterating
4. Basic Operations
   - `append()`
   - + vs. `extend()`
   - `pop()` vs. `remove()` vs. `del()`
5. Strings & Lists
6. Other Operations
7. Aliasing & Cloning
8. Nesting
9. References
10. Acknowledgments

Introduction

In Python, lists

- are ordered sequences of items that can be accessed via index[1]
- are denoted by square brackets, i.e. [ ]

A list contains elements, that can either be:

- homogeneous (e.g., all integers, all strings, ...)
- heterogeneous (less common)

Conversely to strings, lists are mutable:

- elements can be changed

[1] Remember: Python is 0-based!

```
a_list = [ ]              # an empty list
L = [2, 'a', 4, [1,2]]    # an heterogeneous list

len(L)                    # evaluates to 4
L[0]                      # evaluates to 2
L[2] + 1                  # evaluates to 5
L[3]                      # evaluates to [1,2] ... another list
L[4]                      # throws an error

i = 2
L[i-1]                    # evaluates to 'a'

L[1:3]                    # evaluates to ['a', 4]
```

Mutability

Recall

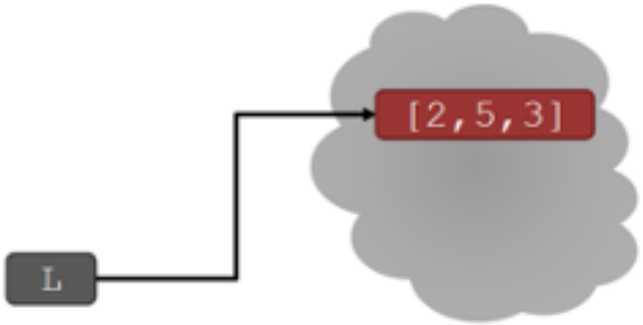
Lists are mutable.

The assignment of an element at a given index changes the value!

```
>>> L = [2, 1, 3]
>>> L[1] = 5
>>> print(L)
[2, 5, 3]
```

Note

This is the same object `L`.



Iterating

Example: compute the sum of elements of a list.
Common strategy? Iterate over the elements of the list.

```
L = [2, 5, 3]
total = 0
for i in range(len(L)):
    total += L[i]
print(total)
```

```
L = [2, 5, 3]
total = 0
for i in L:
    total += i
print(total)
```

Note

- just like strings, you can iterate over list elements directly (right panel)
- list elements are indexed from `0` to `len(L)-1`
- `range(n)` goes from `0` to `n-1`

Basic Operations

Add elements to the end of a list `L` with:

`L.append(<elements>)`

Example:

```
>>> L = [2, 1, 3]
>>> L.append(5)    # lists are objects with data, methods and functions
>>> print(L)
[2, 1, 3, 5]
```

Note

`append()` mutates the list!

Note

Like methods in strings, methods in lists also use the dot notation.

Concatenate lists `L1` and `L2` together with:

`L1.extend(L2)`

or

`L1 + L2`

Example:

```
>>> L1 = [2, 1, 3]
>>> L2 = [4, 5, 6]
>>> L3 = L1 + L2      # L3 = [2, 1, 3, 4, 5, 6], L1 and L2 unchanged
>>> L1.extend(L2)     # L1 = [2, 1, 3, 4, 5, 6], L2 unchanged
```

Note

`extend()` mutates the list, whereas the + operator returns a new list.

You can remove item(s) from a list using three different strategies:

del() delete element at a specific index `i` with `del(L[i])`

pop() delete element at a specific index[2] `i` (and return the removed element) with `L.pop(i)`

remove() remove a specific element `x` with `L.remove(x)`.
Mind the following caveats:

- `L.remove(x)` looks for the element `x` in `L` and, if found, removes it
- if `x` occurs multiple times, removes first occurrence only
- if `x` is not found in `L`, throws an error

Note

All of the three methods mutate the list.

[2] If index is not provided, `L.pop()` deletes the last element of the list by default.

Example (do that in order):

```
>>> L = [2, 1, 3, 6, 3, 7, 0]
>>> L.remove(2)          # now we have L = [1, 3, 6, 3, 7, 0]
>>> L.remove(3)          # now we have L = [1, 6, 3, 7, 0]
>>> del(L[1])            # now we have L = [1, 3, 7, 0]
>>> a = L.pop()          # now we have L = [1, 3, 7] and a = 0
```

Note

- when running `L.remove(3)`, only the first `3` is gone
- when using `L.pop()`, the assignment of the returned value is not mandatory

Strings & Lists

Convert a string `s` to a list `L` using `L = list(s)`

- this yields a list where each element is a character from `s`

Split a string `s` on a character parameter[3] using `s.split()`

- this yields a list where each element is a portion of `s`

Merge a list of characters/strings `L` using `".join(L)`

- this yields a string which sees the concatenation of items in `L` with a given character[4] in-between

[3] Splits on whitespace if called with no parameters.

[4] You can specify a character (between quotes) to add between every element

Examples:

```
>>> s = "I <3 Python"                    # s is a string
>>> L = list(s)                          # L is a list
>>> print(L)
['I', ' ', '<', '3', ' ', 'P', 'y', 't', 'h', 'o', 'n']

>>> L = s.split()        # defaults to whitespace
>>> print(L)
['I', '<3', 'Python']
>>> L = s.split('<')     # custom character
>>> print(L)
['I ', '3 Python']

>>> L = ['Hello', 'World']    # L is a list
>>> s = ''.join(L)            # no character (plain concatenation)
>>> print(s)
'HelloWorld'
>>> s = '_'.join(L)           # custom character in-between
>>> print(s)
'Hello_World'
```

Other Operations

Python has a set of built-in methods that you can use on lists.

You can find the full list at

https://docs.python.org/3/tutorial/datastructures.html

Note

Some list methods return new values. All main list methods mutate the list in-place.

Let's go through the most popular ones, shall we?

- sort elements in a list: `sort()` and `sorted()`
- flip elements last-to-first in a list: `reverse()`

Examples:

```
>>> L = [9, 6, 0, 3]
>>> L1 = sorted(L)    # returns sorted list in L1, does not change L
>>> L.sort()          # L is now mutated (sorted)
>>> print(L)
[0, 3, 6, 9]
>>> print(L1)
[0, 3, 6, 9]
>>> L.reverse()       # L is now mutated (flipped)
>>> print(L)
[9, 6, 3, 0]
```

To mutate or not to mutate, that is the question.

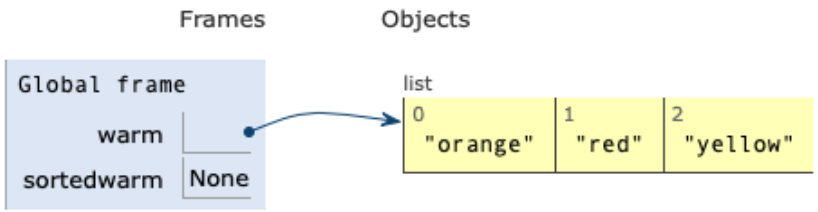
- calling `sort()` mutates the list, returns nothing (`None`)
- calling `sorted()` does not mutate the list, must assign to a new variable

```
warm = ['red', 'yellow', 'orange']
sortedwarm = warm.sort()
print(warm)
print(sortedwarm)
```

```
cool = ['grey', 'blue', 'green']
sortedcool = sorted(cool)
print(cool)
print(sortedcool)
```

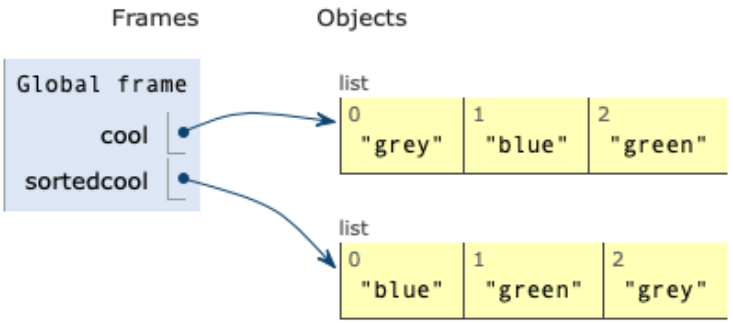
Print output (drag lower right corner to resize)

```
['orange', 'red', 'yellow']
None
```



Print output (drag lower right corner to resize)

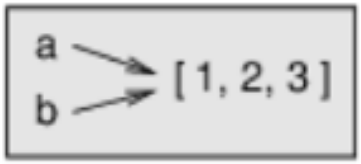
```
['grey', 'blue', 'green']
['blue', 'green', 'grey']
```



Aliasing & Cloning

Aliasing: if `a` refers to an object and you assign `b = a`, then both variables refer to the same object.

```
>>> a = [1, 2, 3]
>>> b = a
>>> b is a
True
```



Reference: association of a variable with an object (in the example above, two references to the same object)

Aliased: an object with more than one reference has more than one name, so we say that the object is *aliased*.

If the aliased object is mutable (like lists), changes made to one alias affect the other:

```
>>> a = [1, 2, 3]
>>> b = a
>>> b[0] = 42
>>> print(b)
[42, 2, 3]
>>> print(a)
[42, 2, 3]
```

Warning

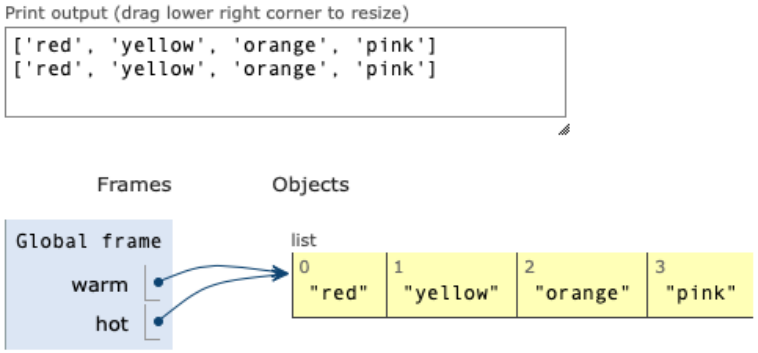
This aspect can be useful, but error-prone!

Hint

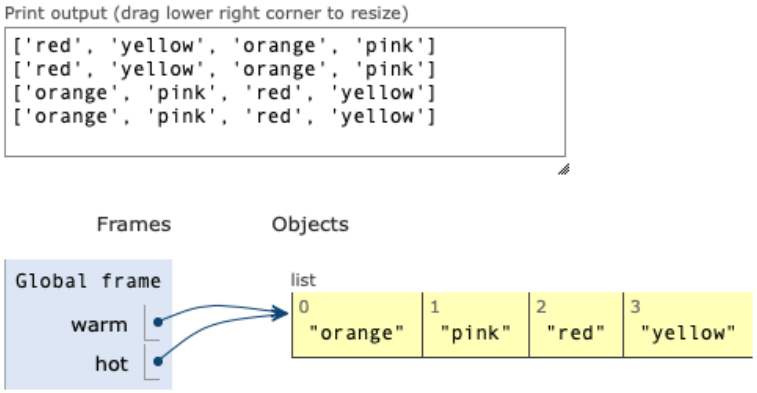
To be safe, avoid aliasing when working with mutable objects.

The aliasing side-effect also holds for list methods we saw earlier:

```
>>> warm = ['red', 'yellow', 'orange']
>>> hot = warm                          # we have an alias here
>>> hot.append('pink')
>>> print(hot)
>>> print(warm)                         # Breakpoint 1
>>> warm.sort()
>>> print(hot)
>>> print(warm)                         # Breakpoint 2
```



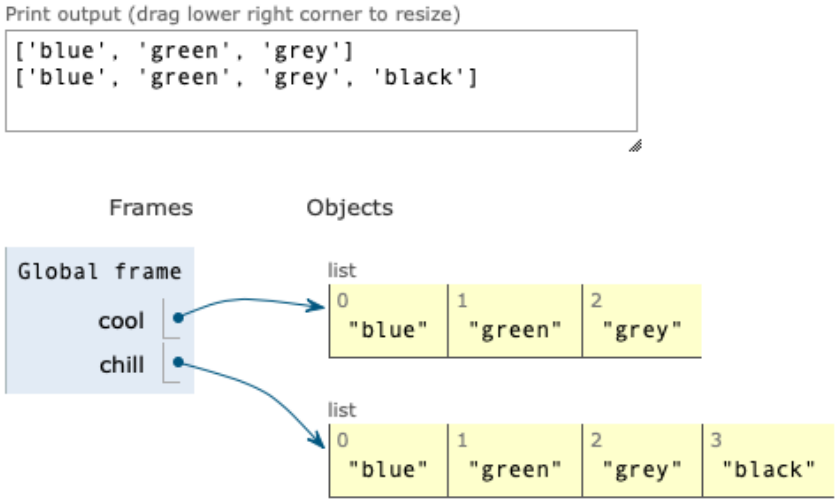
(a) Breakpoint 1



(b) Breakpoint 2

Cloning: create a new list (different object) by copying every element via assignment `b = a[:]`

```
>>> cool = ['blue', 'green', 'grey']
>>> chill = cool[:]                    # we have a clone here
>>> chill.append('black')
>>> print(cool)
>>> print(chill)
```



Warning

Never mutate a list as you iterate through it.

Example: write a function that removes duplicates from two lists.

```
def remove_dups(L1, L2):
    for item in L1:
        if item in L2:
            L1.remove(item)
    return L1
```

```
def remove_dups(L1, L2):
    L1copy = L1[:]
    for item in L1copy:
        if item in L2:
            L1.remove(item)
    return L1
```

```
L1 = [1,2,3,4]
L2 = [1,2,5,6]
print(remove_dups(L1, L2))    # left panel returns [2, 3, 4]
print(remove_dups(L1, L2))    # right panel returns [3, 4]
```

Left panel is incorrect. Why?

- Python uses an internal counter to keep track of the position in the loop
- Mutation changes the length, but Python does not update the counter
- The loop never visits element 2 in L1

Nesting

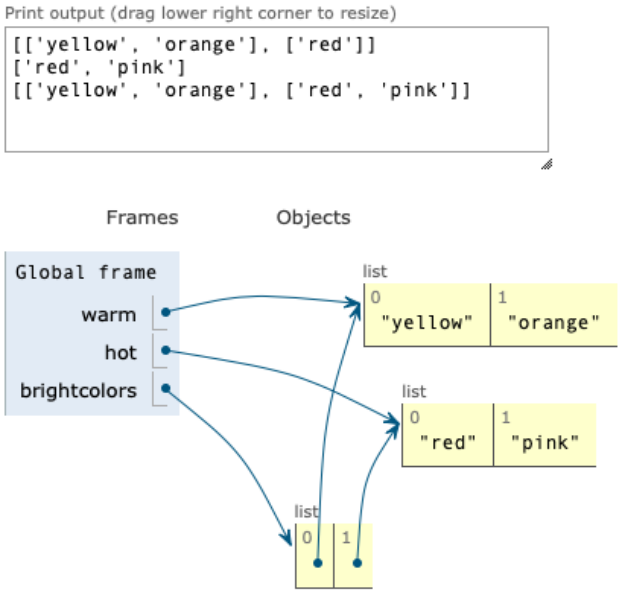
Recall

Lists can be heterogeneous. An element of a list can itself be a list.

Warning

Mutation and aliasing affect also nested lists.

```
warm = ['yellow', 'orange']
hot = ['red']
brightcolors = [warm]
brightcolors.append(hot)
print(brightcolors)
hot.append('pink')
print(hot)
print(brightcolors)
```



References

- Guttag, J. V. (2013). *Introduction to Computation and Programming using Python (Revised and Expanded Edition)*. MIT Press. [Sections 5.2 and 5.4]
- Downey, A. (2015). *Think Python*. Green Tea Press ed. 2nd. [Chapter 10]

Acknowledgments